

LARAVEL PART 2

IT9a/L



Pass Data to Views



Pass Data to Views

- Use the 3rd parameter of `Route::view()` -> **`Route::view('/', 'home', ['greeting' => 'Hello Word!']);`**
- Go to **`home.blade.php`** and echo out the greeting variable -> **`{{ $greeting }}`**;
- Add person variable with variable Andrie -> **`Route::view('/', 'home', ['greeting' => 'Hello', 'person' => 'Andrie',]);`**
- Add the person variable to your home page -> **`{{ $greeting }}`** , **`{{ $person }}`**!



Pass Data to Views

- But what if the name or person should be passed through query strings? (person=Frank)
- use the `request('person');` -> `Route::view('/', 'home', ['greeting' => 'Hello', 'person' => request('person'),]);`



The security of {{ }}

- if we use the traditional php syntax `<?= $person ?>`, it is still going to work but we don't have any protection in place, which means, if we update the query ->
`person=<script>alert('yo')</script>`
- Now, even though we didn't explicitly trigger the alert, the user sees one because of this vulnerability.
- By using the blade syntax, `{{ }}`, we have the protection from this vulnerability.



The security of {{ }}

- By using the blade syntax, `{{ }}`, we have the protection from this vulnerability.
- Any time you define an HTML form in Laravel that makes a POST, PUT, PATCH, or DELETE request, you must include a hidden CSRF token field. If you don't, Laravel's security middleware will reject the request and throw a 419 Page Expired error.



Pass Data to Views

- Now, you don't want to use the view(). you want to use the get() -> ***Route::get('/', function(){return view('home', ['greeting' => 'Hello', 'person' => request('person'),])});***
- It is functionally identical
- Now, to have a default value for the person incase the user didn't put value -> request('person', 'World')



Blade Directives



Blade Directives

- What if you need to pass a list of tasks to your view. ->
Route::get('/', function(){return view('home', ['tasks' => 'Go to the market', 'Walk the dog', 'Watch a video tutorial']);});
- In your home page, if you use ***{{ \$tasks }}***, it will result to an error.
- Usually in php vanilla, you do this -> `<?php var_dump($tasks)?>` or `<?php foreach ($tasks as $task) {echo $task . " ";}?>`
- use the `@dump($tasks)`
- use the `@dd($tasks)` to die and dump
- what if we check if we have tasks



Blade Directives

- in php vanilla we usually do ->

```
<?php if (count($tasks)) { ?>
```

```
<p>Yes we have <?= count($tasks) ?> tasks.</p>
```

```
<?php }?>
```

- in blade template, we use ->

```
@if (count($tasks))
```

```
<p>Yes we have <?= count($tasks) ?> tasks.</p>
```

```
@endif
```



Blade Directives

- If not, use the @unless
- You can also use the @for, @while
- use the @foreach and li elements to output the tasks
- what if the array is empty and you want to output something saying that there are no tasks?
- you can use @if but negate the count, @unless, or use the @empty in the @forelse



Blade Directives

ex.

```
@if (!count($tasks))  
    <p>There are no tasks available.</p>  
@endif
```

```
@unless(count($tasks))  
    <p>There are no tasks available.</p>  
@endunless
```

```
@forelse ($tasks as $task)  
    <li>{{ $task }}</li>  
@empty  
    <p>There are no tasks available.</p>  
@endforelse
```